

Foundations of Transformer architecture deep learning

[Transformer architecture deep learning]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.0 Content Block

Predecessors For many years, sequence modelling and generation was done by using plain recurrent neural networks (RNNs). A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens. A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models. Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence. Modern transformers overcome this problem, but unlike RNNs, they require computation time that is quadratic in the size of the context window. The linearly scaling fast weight controller (1992) learns to compute a weight matrix for further processing depending on the input. One of its two networks has "fast weights" or "dynamic links" (1981). A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.

2.0 Content Block

KV caching When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The KV caching method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching. If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots. In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called prefilling. Hyperscalers serving large Transformer models may use disaggregated inference, wherein prefilling and decoding are performed on separately specialized hardware.

3.0 Content Block

Relative Position Encodings Relative Position Encodings is similar to ALiBi, but more generic: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + B\right)V$ where B is a Toeplitz matrix, that is, $B_{i,j} = B_{i',j'}$

$B_{i,j} = B_{i',j'}$ whenever $i - j = i' - j'$. This is contrasted with the original sinusoidal positional encoding, which is an "absolute positional encoding".

4.0 Content Block

where FFN stands for "feed-forward network". We can more succinctly write it as $\text{EncoderLayer}(H) = \text{FFN}(\text{MultiheadAttention}(H, H, H))$ with the implicit convention that the FFN is applied to each row of the matrix individually. The encoder layers are stacked. The first encoder layer takes the sequence of input vectors from the embedding layer, producing a sequence of vectors. This sequence of vectors is processed by the second encoder, and so on. The output from the final encoder layer is then used by the decoder. As the encoder processes the entire input all at once, every token can attend to every other token (all-to-all attention), so there is no need for causal masking.

5.0 Content Block

FlashAttention FlashAttention is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow). See the page on softmax for details. An improved version, FlashAttention-2, was developed to cater to the rising demand for language models capable of handling longer context lengths. It offers enhancements in work partitioning and parallelism, enabling it to achieve up to 230 TFLOPs/s on A100 GPUs (FP16/BF16), a 2x speed increase over the original FlashAttention. Key advancements in FlashAttention-2 include the reduction of non-matmul FLOPs, improved parallelism over the sequence length dimension, better work partitioning between GPU warps, and added support for head dimensions up to 256 and multi-query attention (MQA) and grouped-query attention (GQA). Benchmarks revealed FlashAttention-2 to be up to 2x faster than FlashAttention and up to 9x faster than a standard attention implementation in PyTorch. Future developments include optimization for new hardware like H100 GPUs and new data types like FP8. FlashAttention-4 focuses on pipelining to increase instruction throughput, and was developed to perform particularly well on Blackwell GPUs.

6.0 Content Block

machine translation time series prediction document summarization document generation named entity recognition (NER) writing computer code based on requirements expressed in natural language. speech-to-text Beyond traditional NLP, the transformer architecture has had success in other applications, such as:

7.0 Content Block

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google. The predecessors of transformers were developed as an improvement over previous architectures for machine translation, but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning, audio, multimodal learning, robotics, playing chess and at disaster responses. It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs) and BERT (bidirectional encoder representations from transformers).